



Unidade 2 — Frameworks



Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Identificar** os componentes internos de um framework;
- **Explicar** a função de cada componente (Router, Controller, Middleware, Service, Repository);
- **Analisar** o fluxo de uma requisição dentro de um framework;
- **Comparar** a estrutura de frameworks Back-End e Front-End.

Taxonomia de Bloom: Identificar, Explicar, Analisar, Comparar.



Componentes Fundamentais

Requisição HTTP



Router

→ Qual handler executar?



Middleware

→ Processar antes do handler?



Controller

→ Receber requisição, devolver resposta



Service

→ Lógica de negócio



Repository

→ Acesso a dados



Response

→ Devolver ao cliente



Router — Roteamento

```
// Definição de rotas
app.get('/api/usuarios', usuarioController.listar);
app.get('/api/usuarios/:id', usuarioController.buscar);
app.post('/api/usuarios', usuarioController.criar);
app.put('/api/usuarios/:id', usuarioController.atualizar);
app.delete('/api/usuarios/:id', usuarioController.excluir);
```

- Mapeia **URL + Método HTTP** para um **handler**;
- Extrai **parâmetros** da URL (`:id`);
- É o "despachante" da aplicação.



Middleware — Pipeline de Processamento

```
// Middleware de parse JSON
app.use(express.json());

// Middleware de autenticação
function autenticar(req, res, next) {
  const token = req.headers.authorization;
  if (!token) {
    return res.status(401).json({ erro: 'Token não fornecido' });
  }
  // Validar token...
  next();
}

// Middleware de log
function logger(req, res, next) {
  console.log(`${req.method} ${req.url} - ${new Date()}`);
  next();
}
```



Middleware — Pipeline de Processamento

- Funções que executam **antes** do handler;
- Podem **interceptar**, **modificar** ou **rejeitar** a requisição;
- Formam uma **pipeline** (cadeia de processamento).



Controller — Orquestração

```
const usuarioController = {
  async listar(req, res) {
    try {
      const usuarios = await usuarioService.listar();
      res.json(usuarios);
    } catch (error) {
      res.status(500).json({ erro: 'Erro interno' });
    }
  },

  async criar(req, res) {
    try {
      const usuario = await usuarioService.criar(req.body);
      res.status(201).json(usuario);
    } catch (error) {
      res.status(400).json({ erro: error.message });
    }
  }
};
```



Controller — Orquestração

- Recebe a **requisição** e devolve a **resposta**;
- **Orquestra** o fluxo: chama o Service, formata a resposta;
- Não contém **lógica de negócio** diretamente.



Service — Lógica de Negócio

```
const usuarioService = {
  async listar() {
    return await usuarioRepository.buscarTodos();
  },

  async criar(dados) {
    // Validação de negócio
    if (!dados.nome || dados.nome.length < 3) {
      throw new Error('Nome deve ter pelo menos 3 caracteres');
    }
    if (!dados.email || !dados.email.includes('@')) {
      throw new Error('E-mail inválido');
    }
    // Verificar se e-mail já existe
    const existente = await usuarioRepository.buscarPorEmail(dados.email);
    if (existente) {
      throw new Error('E-mail já cadastrado');
    }
    return await usuarioRepository.criar(dados);
  }
};
```



Service — Lógica de Negócio

- Contém a **lógica de negócio** da aplicação;
- **Não conhece** HTTP (não usa req, res);
- Recebe **dados** e retorna **dados**.



Repository — Acesso a Dados

```
const usuarioRepository = {
  async buscarTodos() {
    // Pode ser SQL, MongoDB, arquivo, etc.
    return await db.query('SELECT * FROM usuarios');
  },

  async buscarPorEmail(email) {
    return await db.query('SELECT * FROM usuarios WHERE email = ?', [email]);
  },

  async criar(dados) {
    const result = await db.query(
      'INSERT INTO usuarios (nome, email) VALUES (?, ?)',
      [dados.nome, dados.email]
    );
    return { id: result.insertId, ...dados };
  }
};
```



Repository — Acesso a Dados

- **Abstrai** o acesso ao banco de dados;
- O Service **não sabe** se é SQL, NoSQL ou arquivo;
- Facilita a **troca** de banco de dados.



Fluxo Completo no Framework

Requisição: POST /api/usuarios

Body: { "nome": "Maria", "email": "maria@email.com" }

1. Router → identifica POST /api/usuarios → usuarioController.criar
2. Middleware → express.json() → parse do body
3. Middleware → autenticar → verifica token → next()
4. Controller → extrai dados de req.body → chama usuarioService.criar()
5. Service → valida nome, email → verifica duplicidade → chama usuarioRepository.criar()
6. Repository → INSERT INTO usuarios → retorna objeto criado
7. Service → retorna objeto para o Controller
8. Controller → res.status(201).json(usuario)
9. Resposta: 201 Created + JSON com o usuário



Síntese da Aula

Conceitos principais:

1. Todo framework é composto por **componentes fundamentais**: Router, Middleware, Controller, Service, Repository;
2. O **Router** mapeia URLs para handlers;
3. O **Middleware** processa a requisição antes do handler;
4. O **Controller** orquestra o fluxo sem conter lógica de negócio;
5. O **Service** contém a lógica de negócio;
6. O **Repository** abstrai o acesso a dados.



Pergunta reflexiva:

Se cada componente tem uma responsabilidade clara, o que acontece quando um componente "invade" a responsabilidade de outro?

Próxima aula: Frameworks Back-End e Front-End — Vamos conhecer os principais frameworks do mercado.

